

C++ for Loop

In C++, the *for loop* is an entry-controlled loop that is mainly utilized to iterate a part of the program multiple times. If the number of iterations is fixed, it is recommended to use for loop than [while](#) or [do-while](#) loops.

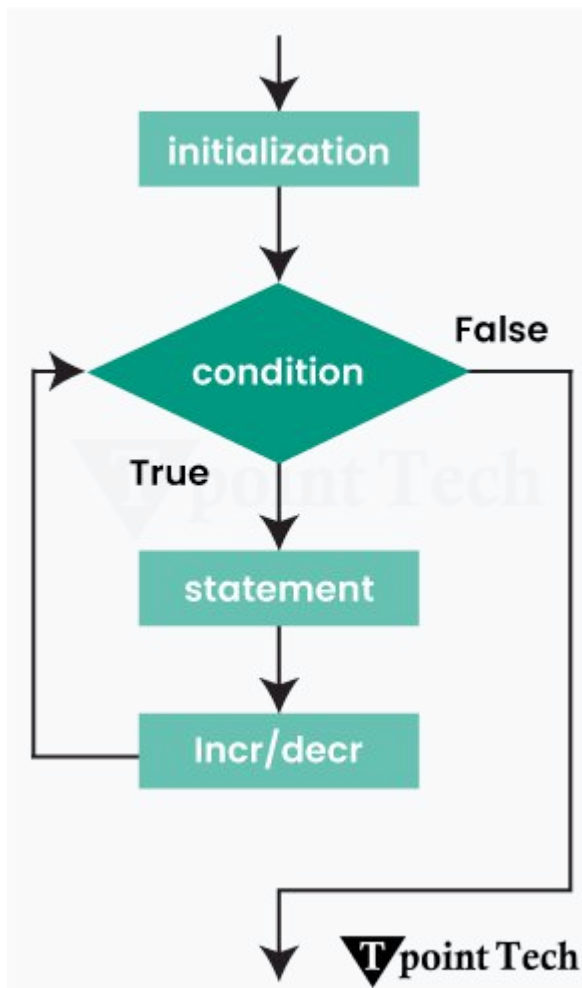
Syntax:

The [C++](#) for loop is the same as other languages, such as [C](#) and [C#](#). We can initialize [variables](#), check conditions, and increment/decrement values.

```
for(initialization; condition; update){  
  
//code to be executed  
  
}
```

- **Initialization:** It initializes the [variables](#) and is executed only once.
- **Condition Statement:** The condition statement checks the given condition. If the condition is true, it executes the of for loop. If the condition is false, it terminates the for loop.
- **Update Expression:** After execution of the loop body, this expression increments or decrements the loop variable by 1.

Flowchart of for Loop:



As we can see in the flowchart, use the following steps to implement the for loop in C++:

Step 1: First, go to the for loop and initialize the variables and their values.

Step 2: Next, the control flow jumps to the condition statement.

Step 3: Here, the condition is tested:

- If the condition is true, it executes the loop body.
- If the condition is false, it terminates the loop body and moves to the next iteration.

Step 4: After that, it goes to the update expression, where it increases or decreases the value by 1, and then again, it goes to step 3 to check the condition.

Step 5: At the end, it exits from the loop and prints the output.

Simple C++ for Loop Example:

Let us take a simple example to print the numbers 1 to 10 using the C++ for loop.

```
#include <iostream>

using namespace std; //using standard namespace

int main() //main function
{
    for(int i=1;i<=10;i++) //code of C++ for loop
    {
        cout<<i <<"\n";
    }
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Factorial Program using C++ for Loop:

Let us take another example to find the factorial of a given number using a for loop in C++.

```
#include <iostream>

using namespace std; //using the standard namespace

// main code

int main() {

    int n;

    int fact = 1;

    cout << "Enter a positive number: ";

    cin >> n;

    // for loop calculating factorial

    for (int i = 1; i <= n; i++) {

        fact = fact * i;

    }

    cout << "The factorial of " << n << " is: " << fact; //print the output

    return 0;

}
```

Compiler

Output:

Enter a positive number: 6

The factorial of 6 is: 720

Multiplication table program using for loop in C++

Let us take an example to print a multiplication table using for loop in C++.

```
#include <iostream>

using namespace std; //using standard namespace

// main function

int main()
{

    int n;

    cout<<"Enter a Number: ";

    cin >> n;

    for (int i = 1; i <= 10; ++i) //for loop

        cout << n << " * " << i << " = " << n * i << endl;

    return 0;

}
```

Compiler

Output:

Enter a Number: 4

4 * 1 = 4

4 * 2 = 8

4 * 3 = 12

4 * 4 = 16

4 * 5 = 20

4 * 6 = 24

4 * 7 = 28

4 * 8 = 32

4 * 9 = 36

4 * 10 = 40

C++ Nested for Loop

In [C++](#), if we have a for loop inside another for loop, it is known as a *nested for loop*. The inner loop is completely executed when the outer loop executes.

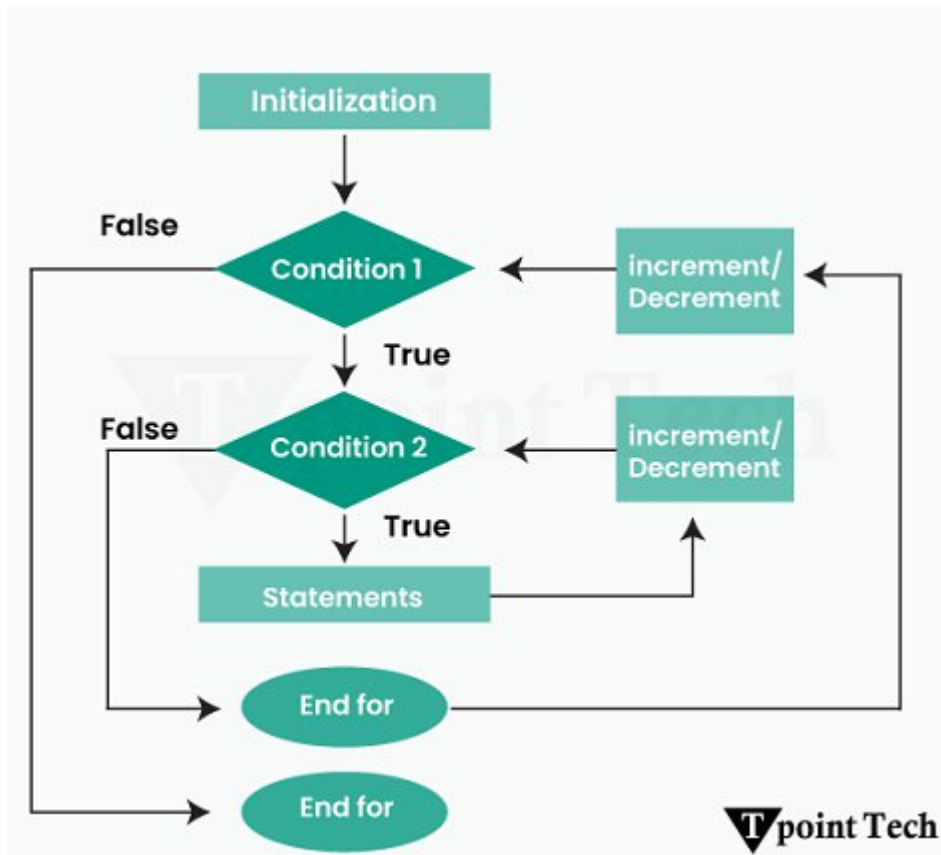
It allows us to perform complex iterations by executing a for loop for every iteration of the outer for loop. Nested for loops are beneficial when working with [multi-dimensional arrays](#) or patterns.

Syntax:

It has the following syntax:

```
for (initialization; condition; increment) {  
    for (initialization; condition; increment) {  
        // statement of inner loop  
    }  
    //statement of outer loop  
}
```

Flowchart of Nested for loop:



C++ Nested for Loop Example:

Let us take a simple example of a nested for loop in C++.

```
#include <iostream>

using namespace std; //using standard namespace

int main () //main function
{
    for(int i=1;i<=3;i++) //outer for loop
    {
        for(int j=1;j<=3;j++) //inner for loop
        {
            cout<<i<<" "<<j<<"\n"; //print the value of i and j
```

```
    }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
2 1  
2 2  
2 3  
3 1  
3 2  
3 3
```

C++ Pyramid Program using Nested for Loop:

Let us take an example to print a pyramid pattern using nested for loop in C++.

```
#include <iostream>  
  
using namespace std;  
  
// main function  
int main()  
{  
    int n;  
    cout << "Enter the Number of rows: ";  
    cin >> n;
```



```
// Outer loop to handle number of rows
for (int i = 0; i < n; i++) {

    // Inner loop to handle number of columns
    for (int j = 0; j <= i; j++) {

        cout << "* "; // Printing stars
    }

    cout << endl; // moves to the new line after every row
}
}
```

Compiler

Output:

Enter the Number of rows: 6

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Explanation:

In this example, we have taken two for loops. Here, the first loop handles the number of rows, and the second loop handles the number of columns and prints the stars.

C++ Inverted Pyramid Program using Nested for Loop:

Let us take an example to print an inverted pyramid pattern using nested for loop in C++.

```
#include <iostream>

using namespace std;

// main function
int main()
{
    int n;

    cout << "Enter the Number of rows: ";

    cin >> n;

    // Outer loop to handle number of rows
    for (int i = 0; i < n; i++) {

        // Inner loop to handle number of columns
        for (int j = n; j > i; j--) {

            cout << "* "; // Printing stars
        }

        cout << endl; // moves to the new line after every row
    }
}
```

Compiler

Output:

Enter the Number of rows: 5

* * * * *

* * * *

* * *

* *

*

Explanation:

In this example, we have taken two for loops. Here, the outer loop handles the number of rows, and the inner loop handles the number of stars per row.

C++ foreach Loop

The foreach loop is also known as the range-based for loop. It is mainly utilized to iterate over the elements of a container (such as [list](#), [array](#), [vector](#), etc) in a simple way without performing initialization, increment/decrement, and testing. The 'for' keyword is utilized for the foreach loop in C++.

Syntax:

It has the following syntax:

```
for (data_type variable_name : container_name) {  
  
    // code to be executed  
  
}
```

In this syntax,

- **Variable_name:** It is a variable name that is provided to the elements stored in a container.
- **Container_name:** It is a variable name of a container of any type, including list, array, vector, and many others.

C++ Foreach Loop Example:

Let us take an example to illustrate the foreach loop in C++.

```
#include <iostream>

#include <vector> //vector library

using namespace std; //using standard namespace

int main() //main function
{
    vector<int> numbers = {15, 20, 25, 30, 35};

    // Range-based for loop to modify each element
    for (int& num : numbers) {
        num += 5; // add 5 for each number
    }

    for (int num : numbers) {
        cout << num << " "; //prints the output
    }
    cout << endl;

    return 0;
}
```

Compiler

Output:

```
20 25 30 35 40
```

Explanation:

In this example, we have taken a vector library for using dynamic arrays. Next, we declare a vector of integers. After that, we use two for loops in this program. The first loop uses a reference to modify the vector elements, and the second loop prints the modified values.

C++ Infinite for Loop

In C++, if we remove the initialization, condition, and update expression, it will result in an infinite loop.

In other words, if we use a double semicolon in the for loop, it will be executed infinite times.

Syntax:

It has the following syntax:

```
for ( ; ; ){  
    //code to be executed  
}
```

C++ Infinite For Loop Example:

Let us take an example to illustrate the infinite for loop in C++.

```
#include <iostream>  
  
using namespace std;  
  
int main () //main function  
{  
    for ( ; ; ) //infinite for loop  
    {  
        cout<<"Hello! Tpointtech"; //prints the output  
    }  
}
```

```
}
```

Compiler

Output:

```
Hello! Tpointtech
```

```
Hello! Tpointtech
```

```
Hello! Tpointtech
```

```
Hello! Tpointtech
```

Here, we need to press Ctrl+C to exit from the infinitive program.

C++ for Loop MCQs

1. In for loop, which of the following operators is utilized to separate expressions in C++?

- a) Colon (:)
- b) Semicolon (;)
- c) Ampersand (&)
- d) Comma (,)

Answer: (b) Semicolon (;)

2. Which of the following options is the first expression of for loop in C++?

- a) Condition
- b) Increment/decrement
- c) Initialization
- d) None of the above

Answer: (c) Initialization

3. What will be the output of the following code?

```
int main() {  
  
    for (int i = 1; i <= 5; i++) {  
  
        cout << i << " ";  
  
    }  
}
```

```
return 0;
```

```
}
```

- a) 1 2 3 4 5
- b) 1 2 3 4 5 6
- c) 1 2 3 4
- d) 0 1 2 3 4 5

Answer: (a) 1 2 3 4 5

4. What type of loop is the for loop in C++ programming?

- a) Exit-controlled loop
- b) Recursive loop
- c) Infinite loop
- d) Entry-controlled loop

Answer: (d) Entry-controlled loop

5. Can we utilize the multiple initialization and update expression in a for loop in C++?

- a) Yes
- b) Only in C
- c) No
- d) Only in while loops

Answer: (a) Yes